

**Universidade do Minho**  
Escola de Engenharia

## ***Projeto de Sistemas Distribuídos***

*3º ano Licenciatura Engenharia Informática*

*Sistemas Distribuídos*

*Universidade do Minho*

*09/01/2026*

Grupo 11

Diogo Alves Ferreira (A106904)

Hugo Araújo Cunha (A106808)

José Miguel da Silva Santos (A72443)

Mariana Vivas Rodrigues (A106898)

# Índice

|                                                |          |
|------------------------------------------------|----------|
| <b>1.Introdução</b>                            | <b>3</b> |
| <b>2.Arquitetura e Implementação</b>           | <b>4</b> |
| 2.1. Entidades de Dados                        | 4        |
| 2.2. Organização Modular do Servidor           | 4        |
| 2.3. Middleware de Comunicação                 | 5        |
| 2.4. O Stub                                    | 5        |
| 2.5. Protocolo de Comunicação (Tags e OpCodes) | 5        |
| 2.6. Imagem a representar uma Search           | 6        |
| <b>3. Testes</b>                               | <b>7</b> |
| 3.1. Tester                                    | 7        |
| 3.2. Teste de Carga e Cache                    | 7        |
| <b>4.Conclusão e IA</b>                        | <b>9</b> |
| 4.1. Uso de IA                                 | 9        |
| 4.2. Conclusão                                 | 9        |

# 1.Introdução

Neste projeto, foi desenvolvido um sistema cliente-servidor para gestão e agregação de vendas em séries temporais. O objetivo principal foi criar um servidor robusto capaz de lidar com múltiplos clientes concorrentes, garantindo a integridade dos dados e a eficiência no acesso à memória.

O sistema permite a inserção de vendas, consultas agregadas (somas, médias, máximos) sobre janelas temporais e, crucialmente, suporta notificações de eventos complexos baseados em padrões de vendas (sequenciais e simultâneos).

Um dos maiores desafios abordados foi a gestão eficiente de recursos, implementando uma estratégia de persistência em disco com uma Cache LRU (Least Recently Used) para garantir que o sistema opera dentro de limites de memória predefinidos ( $1 < S < D$ ), mesmo lidando com grandes volumes de dados históricos.

## 2. Arquitetura e Implementação

A arquitetura do projeto foi concebida com base nos conceitos lecionados em aula e consolidada através de discussões preliminares do grupo. Embora as ideias do grupo tenham surgido de forma um pouco dispersa, essa separação acabou por nos ajudar a organizar melhor o trabalho e a cumprir todos os requisitos do projeto.

### 2.1. Entidades de Dados

O sistema baseia-se em duas estruturas de dados fundamentais que suportam a lógica de negócio:

**Utilizadores (Users):** A autenticação é gerida através de um `Map<String, String>` persistente, onde a chave é o username e o valor é a password do utilizador. Esta estrutura é carregada para a RAM (`Memoria.java`) no arranque e atualizada em disco (`users.dat`) a cada novo registo, garantindo persistência imediata. O acesso a este mapa é protegido pelo `lockAuth` para permitir autenticações concorrentes.

**Vendas (Venda):** A classe `Venda` é a unidade atômica do sistema e serializável, definida assim com:

- `timestamp (long)`: Momento da transação.
- `produto (String)`: Identificador do item.
- `quantidade (int)`: Volume transacionado.
- `Preço (int)`: Valor unitário.

**Organização em Memória:** As vendas não estão numa lista única. Para otimizar as agregações temporais, estão organizadas hierarquicamente num `Map<Integer, Map<String, List<Venda>>>`. O primeiro nível representa o Dia (ID) e o segundo nível agrupa a lista de Vendas por Produto. Isto permite que consultas sobre um produto específico num dia específico sejam operações quase instantâneas ( $O(1)$ ).

## 2.2. Organização Modular do Servidor

O código do servidor foi dividido em três componentes principais para evitar uma classe muito grande:

1. **Memoria.java (Gestão de Dados)**: Responsável pelo armazenamento (RAM e Disco). Implementa a Cache LRU que mantém apenas os S dias mais recentes em memória e utiliza o padrão Lazy Loading (os dias só são carregados do disco quando necessários). Inclui ainda um Shutdown Hook para limpeza automática de ficheiros temporários.
2. **GestorCondicoes.java (Lógica de Eventos)**: Isola a complexidade matemática das notificações. Implementa a lógica de "Janela Deslizante" para detetar vendas consecutivas e verificações de variação (Deltas) para vendas simultâneas.
3. **DataBase.java (Coordenação)**: Atua como a central de operações. Gere os ReentrantLocks e Condition Variables para garantir a segurança entre threads e coordena a interação entre a memória e a lógica de eventos.

## 2.3. Middleware de Comunicação

A camada de rede foi construída sobre Sockets TCP, utilizando primitivas fornecidas na unidade curricular (TaggedConnection e Demultiplexer) para abstrair a complexidade da concorrência e serialização, ambos foram-nos fornecidos nas aulas teórico-práticas.

## 2.4. O Stub

O Stub implementa o padrão de desenho Stub. Ele expõe os métodos locais (ex: inserir(), consultarTotal()) e traduz essas chamadas para mensagens de rede. O fluxo de uma operação no Stub é:

1. **Serialização**: Escreve o Código da Operação (OpCode) e os argumentos num buffer de bytes (DataOutputStream).
2. **Envio**: Envia a frame através do Demultiplexer (que usa a TaggedConnection).
3. **Espera**: Bloqueia à espera da resposta correspondente àquela Tag.

4. **Deserialização:** Converte os bytes recebidos de volta para tipos Java (ex: long, String ou List<Venda>) e retorna ao cliente.

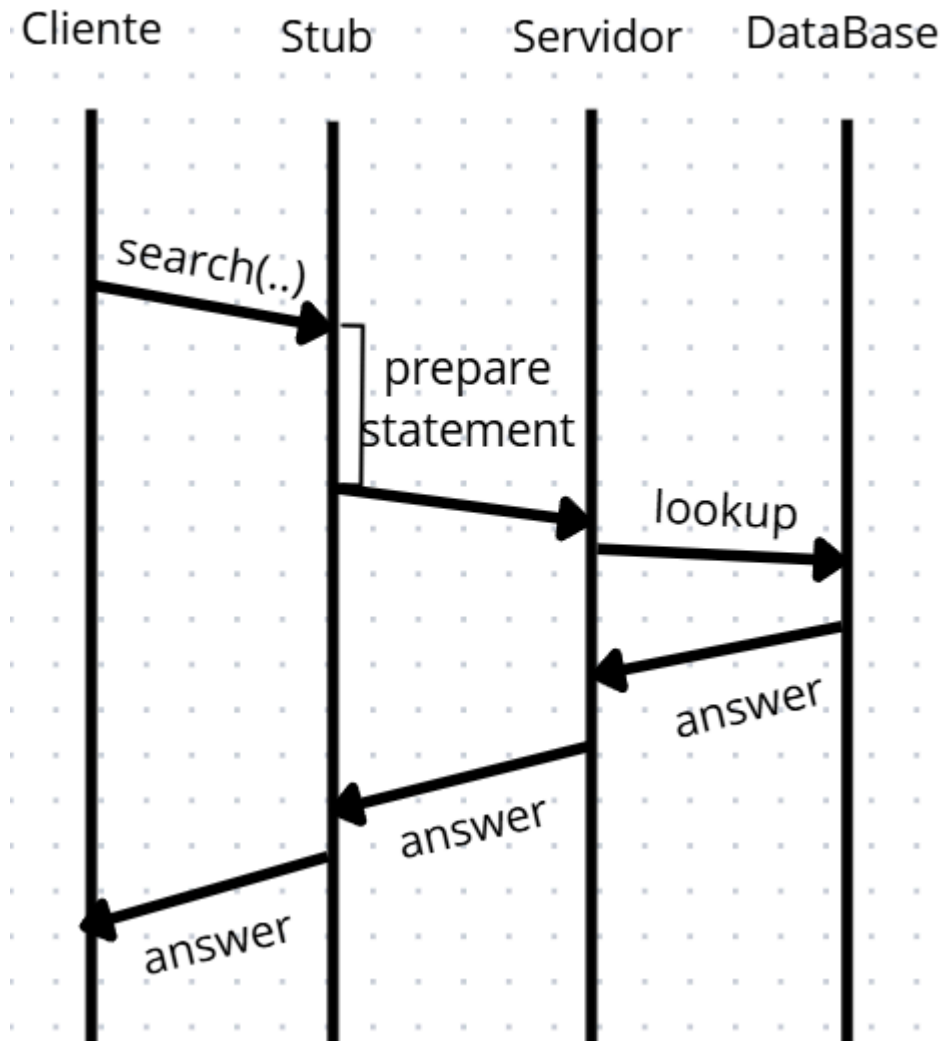
## 2.5. Protocolo de Comunicação (Tags e OpCodes)

A comunicação segue um protocolo binário. O primeiro inteiro do payload de cada mensagem define a operação a realizar (OpCode). Abaixo descreve-se o significado de cada valor utilizado na "Tag de Operação":

| OpCode (Int) | Operação        | Argumentos Enviados (Request)                              | Resposta Esperada (Reply) |
|--------------|-----------------|------------------------------------------------------------|---------------------------|
| 7            | REGISTAR        | User (String), Pass (String)                               | Sucesso (Boolean)         |
| 0            | LOGIN           | User (String), Pass (String)                               | Sucesso (Boolean)         |
| 1            | INSERIR         | Timestamp (Long), Produto (String), Qtd (Int), Preço (Int) | Confirmação (Ack)         |
| 2            | CONSULTAR_TOTAL | Produto (String)                                           | Total acumulado (Long)    |
| 4            | AGREGACAO       | Produto (String), Dias (Int), Tipo (Int)                   | Resultado (Double)        |
| 5            | EVENTOS_DIA     | ListaProdutos (Set), Dia (Int)                             | Lista de Vendas (List)    |
| 8            | ESPERAR_SIMULT  | Produto1 (String), Produto2 (String)                       | Sucesso (Boolean)         |

|   |                |                  |                           |
|---|----------------|------------------|---------------------------|
| 9 | ESPERAR_CONSEC | Quantidade (Int) | Produto detetado (String) |
| 3 | AVANCAR_DIA    | (Sem argumentos) | Confirmação (Ack)         |

## 2.6. Imagem a representar uma Search



## 3. Testes

Foram desenvolvidos dois conjuntos de testes para validar a solução:

### 3.1. Teste

Simula 50 clientes concorrentes a realizar operações aleatórias (inserções e consultas), validando a estabilidade do servidor e a correção dos cálculos agregados.

```
>>> A iniciar Teste de Carga...
--- Resumo de Eventos ---
Produto          | Preço    | Qtd Total  | N° Vendas
-----
Laranja          | 10       | 1013       | 1013
TV               | 10       | 983        | 983
Maca             | 10       | 999        | 999
Radio            | 10       | 931        | 931
Banana           | 10       | 1038       | 1038

=== RESULTADOS ===
Tempo Total: 1.241s
Pedidos Sucesso: 5000
Pedidos Erro: 0
Throughput: 4029.0088638195 ops/sec

Process finished with exit code 0
```

A parte interessante deste Teste é que quando ele faz consultas que não estão em cache ele fica significativamente mais lento, como por exemplo a partir do 3 dia do servidor.

### 3.2. Teste de Carga e Cache

Desenhado para provar a gestão de memória. Configura o servidor com Cache minúscula (S=3) e insere dados de 6 dias. Valida que consultarTotal percorre o histórico sem estourar a memória. Valida a eficácia da Cache LRU medindo o tempo de resposta do disco vs. RAM (Cache Hit), confirmando que a segunda leitura é drasticamente mais rápida.

```

=== INÍCIO TESTE DE CARGA E CACHE ===
>>> A iniciar DB com Cache Size = 3
Simulando Dia 0...[Novo Dia] Arquivando dia 0
[OK]
Simulando Dia 1...[Novo Dia] Arquivando dia 1
[OK]
Simulando Dia 2...[Novo Dia] Arquivando dia 2
[OK]
Simulando Dia 3...[Novo Dia] Arquivando dia 3
[OK]
Simulando Dia 4...[Novo Dia] Arquivando dia 4
[OK]
Simulando Dia 5...[Novo Dia] Arquivando dia 5
[OK]
Povoamento concluído em 153ms.
Estado Esperado da Cache (LRU): Deve conter os dias mais recentes (ex: 3, 4, 5).

>>> TESTE 1: Consultar Total (Scan Completo)
Total Calculado: 16412 (Tempo: 63ms)
VALIDAÇÃO: Se o tempo foi baixo e não viste logs de 'Cache Eviction' massivo, funcionou.

>>> TESTE 2: Consultar | Agregação Dia Antigo (Dia 1)
(Isto deve forçar uma leitura de disco e atualizar a cache)
Tempo da 1ª consulta (Frio/Disco): 16ms

>>> TESTE 3: Repetir Agregação Dia 1 (Cache Hit)
Tempo da 2ª consulta (Quente/RAM): 0ms

[SUCESSO] A Cache funcionou! A segunda leitura foi muito mais rápida.

```

Neste teste confirmamos efetivamente que a cache de facto está a funcionar.

### 3.3. Teste de Robustez

Adicionalmente, implementou-se um teste de "Cliente Lento/Malicioso" para verificar o comportamento do servidor perante problemas de controlo de fluxo TCP.

- **Cenário:** Um cliente envia pedidos contínuos mas nunca lê as respostas do socket, forçando o preenchimento dos buffers TCP e bloqueando a thread do servidor responsável por esse cliente (na operação de escrita).
- **Resultado:** Verificou-se que, graças à arquitetura Thread-per-Client e ao facto de as operações de escrita no socket serem realizadas fora das

secções críticas (locks da base de dados), os restantes clientes continuam a ser servidos normalmente, garantindo a disponibilidade do sistema.

```
=== TESTE DE ROBUSTEZ (CLIENTE LENTO/MUDO) ===
```

```
Cenário: Um cliente envia muitos pedidos mas NUNCA lê as respostas.
```

```
Objetivo: O buffer TCP enche, a thread do servidor bloqueia no 'write'.
```

```
Sucesso se: O 'Cliente Bom' conseguir trabalhar normalmente enquanto o 'Mau' está pendurado.
```

```
>>> A esperar 2 segundos para o buffer encher...
```

```
[Mau] A conectar...
```

```
[Mau] A bombardear o servidor (sem ler respostas)...
```

```
[Mau] Morreu (Provavelmente o servidor fechou a conexão): Uma ligação estabelecida foi anulada pelo software no computador
```

```
>>> [Bom] A tentar conectar e fazer uma operação...
```

```
[Bom] SUCESSO! Consegui inserir uma venda.
```

```
CONCLUSÃO: O servidor é ROBUSTO. O bloqueio de um cliente não afetou o outro.
```

```
Process finished with exit code 0
```

## 4. Conclusão e IA

### 4.1. Uso de IA

Durante o desenvolvimento, foram utilizadas ferramentas de IA para completar código e complementar o pensamento. A utilização focou-se em:

- Depuração de Concorrência: Identificação de falhas na lógica de bloqueio das vendas consecutivas e sugestão da implementação de IDs temporais/deltas para garantir que as threads só acordam com eventos novos.
- Refatorização: Apoio na separação da lógica em classes especializadas (Memoria, GestorCondições), melhorando a legibilidade e inicialmente sobre o projeto e como devia ser dividido.
- Otimização: Discussão sobre a estratégia de Cache Eviction para garantir o cumprimento estrito dos requisitos de memória em certas operações.

A implementação final e a integração dos componentes foram realizadas pelo grupo.

### 4.2. Conclusão

Este projeto consolidou os nossos conhecimentos sobre Sistemas Distribuídos, focando-se na gestão eficiente de estado partilhado e comunicação.

No que toca à concorrência, aprendemos que mecanismos robustos como ReentrantLock e Condition Variables são superiores ao simples synchronized para lógicas complexas. A implementação das notificações provou que evitar Busy Waiting e validar o estado com relógios lógicos é essencial para prevenir Race Conditions e garantir a integridade dos dados em ambientes de alta carga.

Relativamente à comunicação e memória, a abstração da rede via Stub e Tagged Connection simplificou o fluxo de dados, escondendo a complexidade do TCP. Além disso, a implementação manual da persistência com Cache LRU demonstrou na prática o impacto da latência de disco. O desafio de manter o limite  $S < D$  ensinou-nos a equilibrar o desempenho da RAM com a escalabilidade do armazenamento.